



Improving GCG

Ege Çakar Hannah Guan Kayden Kehe

Harvard School of Engineering and Applied Science



Problem & Motivation

- Modern LLMs show brittle alignment despite strong safety training.
- Greedy Coordinate Gradient (GCG)-based Search has been shown to effectively jailbreak aligned models.
- But GCG is slow and targets the output instead of the internal refusal mechanism.

We ask: *Can we improve GCG by explicitly targeting a model's internal refusal direction and making its optimization more efficient?*

Activation-Guided GCG

Goal: optimize an adversarial suffix using GCG to **target the model's internal refusal direction**

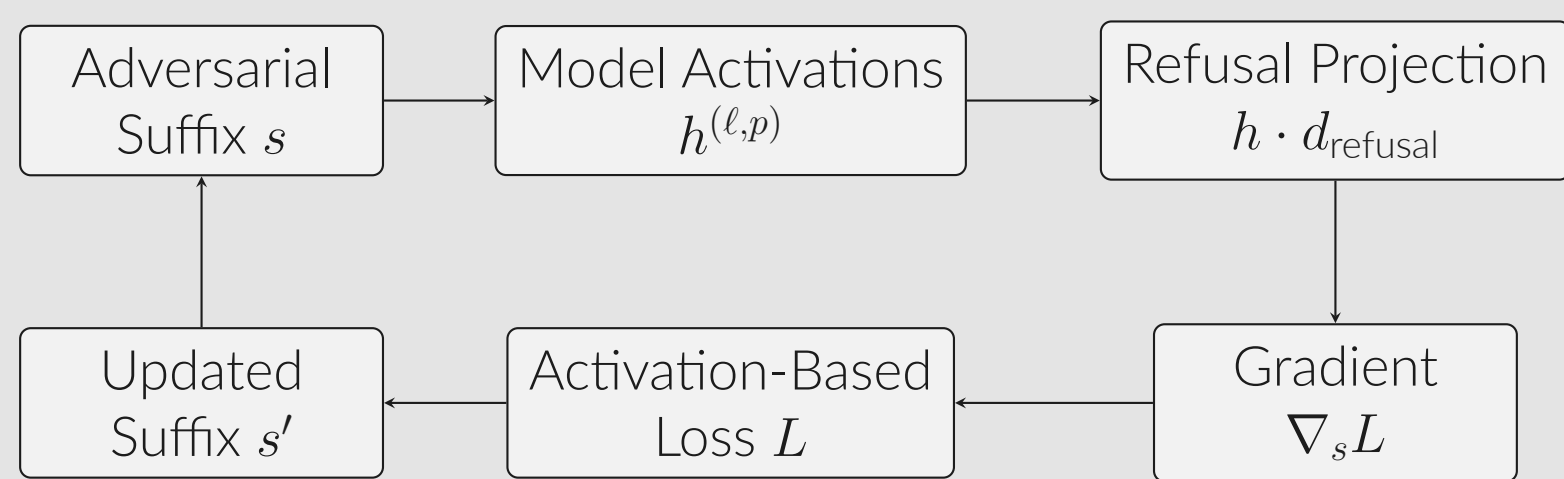


Figure 1. Activation-Based Optimization Loop

Refusal direction: Per-layer refusal directions are calculated as

$$r^{(\ell)} = \mu_{\text{harmful}}^{(\ell)} - \mu_{\text{safe}}^{(\ell)}$$

where $\mu^{(\ell)}$ is the mean activation at layer ℓ .

Activation-based losses: We develop a family of activation-based losses which encourage the suffix to suppress the model's refusal direction:

- Single: Minimize projections at the most influential layer and position.
- Negative: Push the projections at the most influential layer to be negative.
- Layer: Minimize projections across all positions at the most influential layer.
- Token: Minimize projections at a single token position across all layers.
- All: Minimize projections across all layers and positions.

Soft GCG

Goal: speed up GCG by improving the quality of its initialization via **hybrid (continuous then discrete) optimization**

Hybrid Optimization:

- Optimize over a linear composition of the model's vocabulary instead of discretely or directly in the embedding space
- Either use the initialized suffix by Soft-GCG, or continue discrete GCG as usual

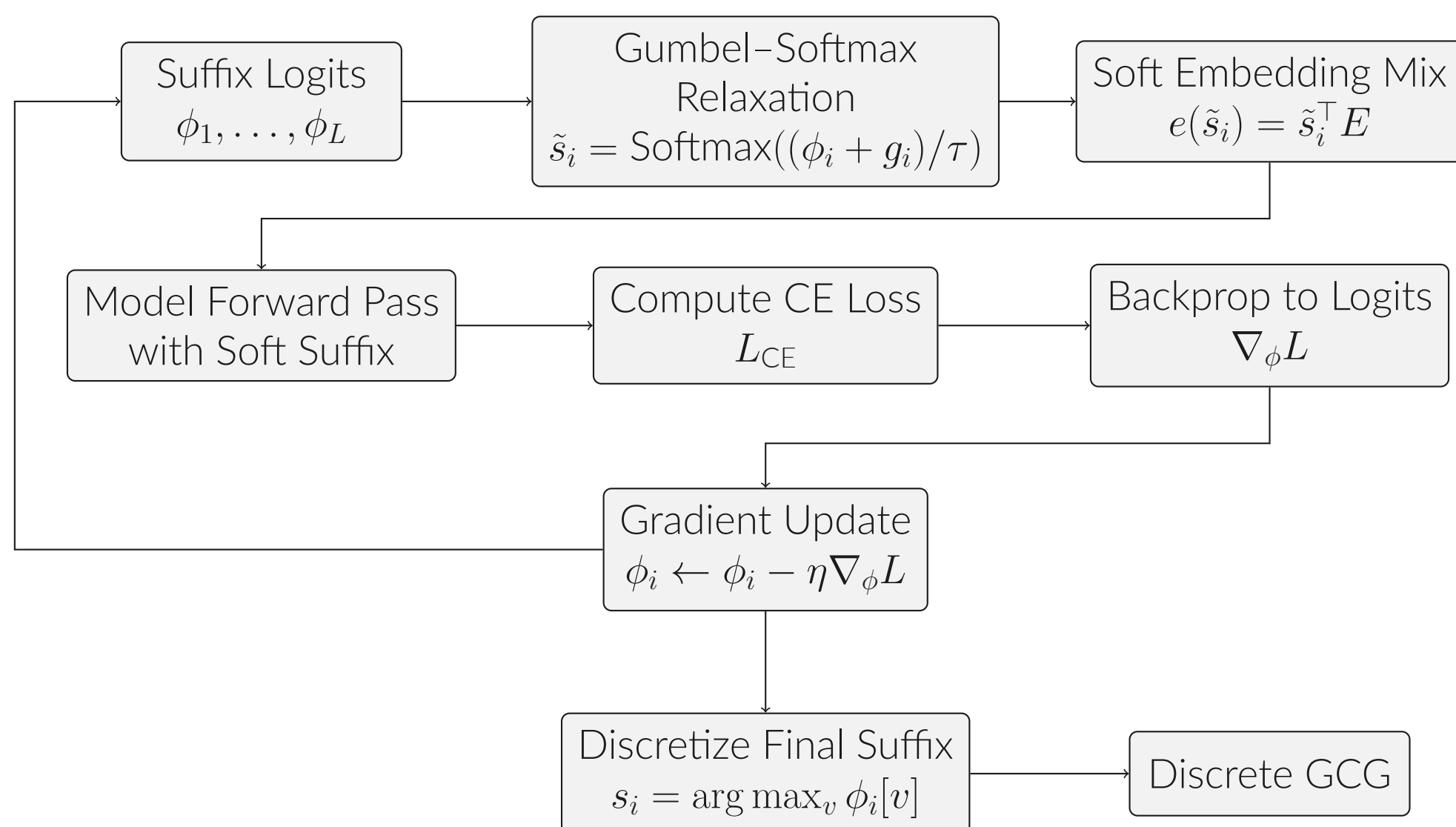
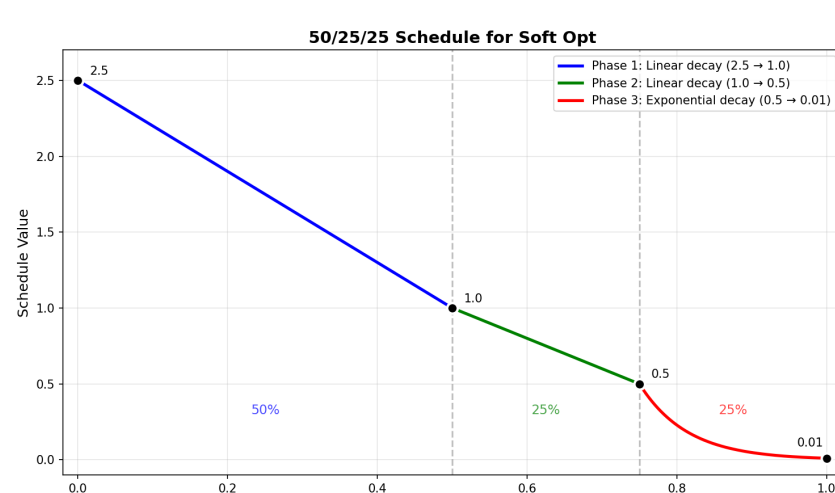


Figure 2. Soft-GCG Optimization Flowchart

Temperature Annealing: Decrease temperature throughout training to move from the soft token regime to the hard token regime. We use a) linear annealing and b) a “slushy” annealing.



Evaluation Metrics

How do we define “harmfulness” for evaluation?

- Substring ASR: Labels a completion as harmful if it does *not* contain any of a fixed list of refusal-style prefixes (e.g., “I’m sorry”, “I cannot”).
- LlamaGuard2 ASR: Labels a completion as harmful if it is labeled unsafe by the LlamaGuard2 safety classifier.
- HarmBench ASR: Labels a completion as harmful if it is labeled unsafe by the HarmBench evaluation framework.

Experiment Settings

Datasets:

- Activation-Guided GCG is tested on the test split of the harmful data from Arditi et al. (2024).
- Soft GCG is tested on the AdvBench dataset, which contains paired (goal, target) strings describing harmful behaviors and their desiderata.

Activation-Guided GCG Settings:

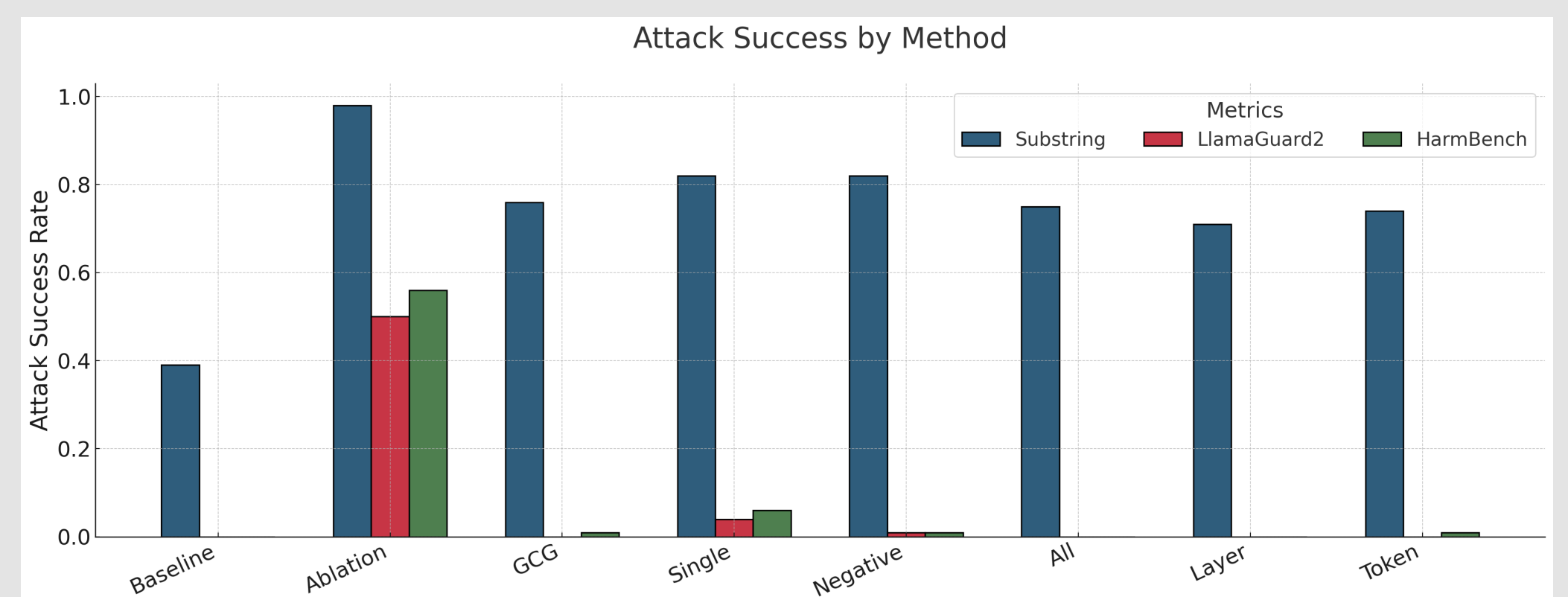
- suffix length fixed at 20 tokens
- 200 optimization steps

Soft GCG Settings:

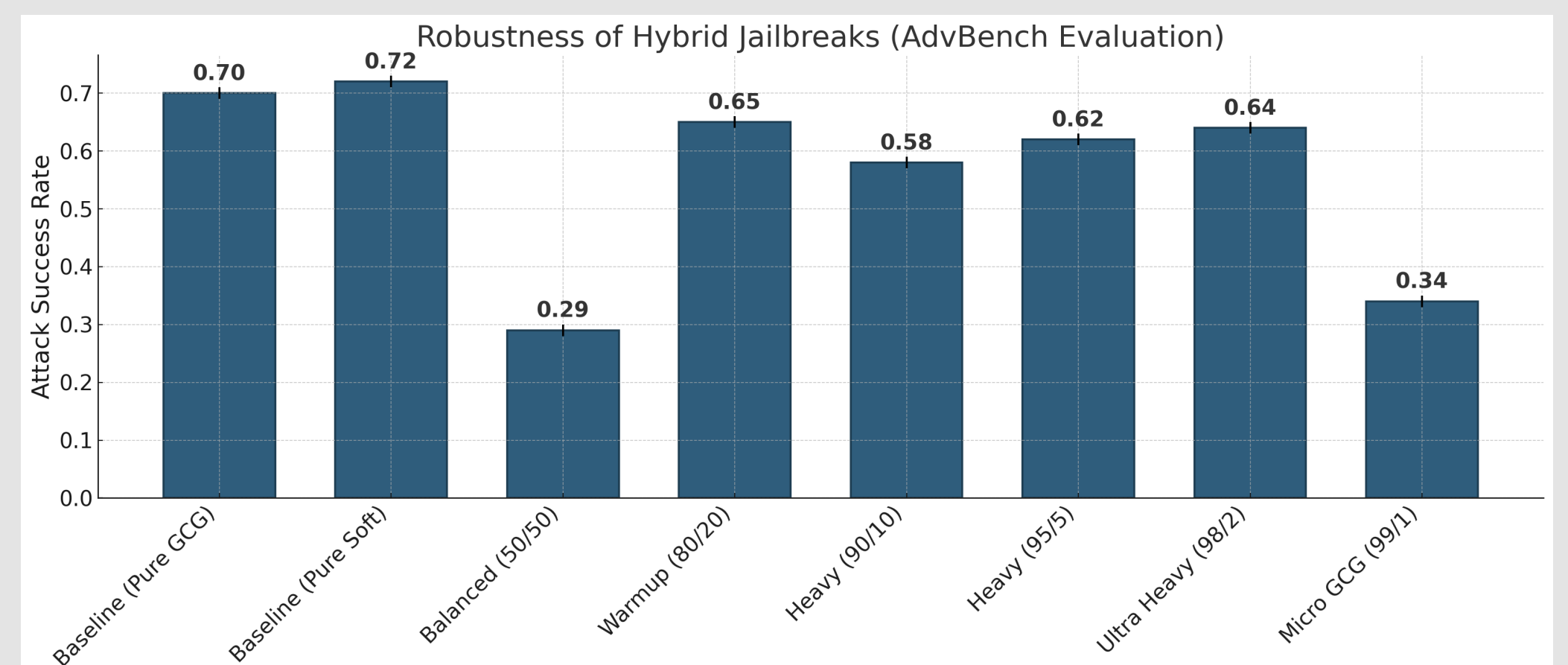
- suffix length fixed at 20 tokens
- Gemma experiments use 500 steps of continuous optimization and batch size 10
- Adam learning rate is 0.1

Experiment Results

Activation-based objectives show improved ASR using less optimization.



Soft GCG on its own achieves performance parity with GCG while achieving a 43x speedup in running time.



Soft GCG achieves meaningful attack success on capable modern models.

